SIMATS SCHOOL OF ENGINEERING

Approved By AICTE | IET-UK Accreditation

Course Code: DSA03	Course Name: Natural Language Processing
Branch: CSE	Year : 2026

Name: R.Rohith Manikanta
Reg no: 192425326

Assignment 2

1. Figure 4.1 shows the bigram counts from a piece of a bigram grammar from the Berkeley Restaurant Project. Note that the majority of the values are zero. Perform the add-1 smoothed counts for the below bigram probability values. (10M) (CO2) (BLT4)

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Figure 4.1 Bigram counts for eight of the words (out of $V = 1446$) in the Berkeley Restaurant Project corpus of 9332 sentences.

2. Suppose someone took all the words in a sentence and reordered them randomly. Write a program which take as input bag of words such a bag of words and produces as output a guess at the original order. You will need to an N-gram grammar produced by your N-gram program (on some corpus. This task is sometimes called bag generation. (10M) (CO2) (BLT4)

3. Use the Penn Treebank tagset to tag each word in the following sentences from Damon Runyon's short stories. You may ignore punctuation. Some of these are quite difficult; do your best. (10M) (CO2) (BLT4)

   a. It is a nice night.

   b. This crap game is over a garage in Fifty-second Street. . .

   c. . . .Nobody ever takes the newspapers she sells . . .

   d. He is a tall, skinny guy with a long, sad, mean-looking kisser, and a mournful voice. e. . . . I am sitting in Mindy's restaurant putting on the gefillte fish, which is a dish I am very fond of, . . .

   f. When a guy and a doll get to taking peeks back and forth at each other, why there you are indeed.

Answer 1. Add-1 (Laplace) smoothed bigram counts and probabilities

Add-1 smoothing adds one to every bigram count, so the zero entries in Figure 4.1 stop being zero. With vocabulary size V = 1446, the smoothed probability and the smoothed (reconstituted) count are

$$P^*(w_n \mid w_{n-1}) = (C(w_{n-1}\, w_n) + 1) / (C(w_{n-1}) + V)$$
$$C^*(w_{n-1}\, w_n) = (C(w_{n-1}\, w_n) + 1) \times C(w_{n-1}) / (C(w_{n-1}) + V)$$

The unigram counts needed for the denominators come from the same Berkeley Restaurant corpus (they are not printed in Figure 4.1): C(i)=2533, C(want)=927, C(to)=2417, C(eat)=746, C(chinese)=158, C(food)=1093, C(lunch)=341, C(spend)=278.

Worked examples. C*(i want) = (827+1) × 2533 / (2533+1446) = 828 × 2533 / 3979 = **527.1**; P*(want | i) = 828 / 3979 = **0.2081**. For a zero count, C*(i to) = 1 × 2533 / 3979 = **0.64** and P*(to | i) = 1 / 3979 = **0.00025**.

Table 1. Add-1 smoothed bigram counts C* (rows = previous word, columns = next word)

	i	want	to	eat	chinese	food	lunch	spend
i	3.8	527.1	0.6	6.4	0.6	0.6	0.6	1.9
want	1.2	0.4	237.9	0.8	2.7	2.7	2.3	0.8
to	1.9	0.6	3.1	429.8	1.9	0.6	4.4	132.6
eat	0.3	0.3	1.0	0.3	5.8	1.0	14.6	0.3
chinese	0.2	0.1	0.1	0.1	0.1	8.2	0.2	0.1
food	6.9	0.4	6.9	0.4	0.9	2.2	0.4	0.4
lunch	0.6	0.2	0.2	0.2	0.2	0.4	0.2	0.2
spend	0.3	0.2	0.3	0.2	0.2	0.2	0.2	0.2

Table 2. Add-1 smoothed bigram probabilities P*(column | row)

	i	want	to	eat	chinese	food	lunch	spend
i	0.0015	0.2081	0.0003	0.0025	0.0003	0.0003	0.0003	0.0008
want	0.0013	0.0004	0.2566	0.0008	0.0029	0.0029	0.0025	0.0008
to	0.0008	0.0003	0.0013	0.1778	0.0008	0.0003	0.0018	0.0549
eat	0.0005	0.0005	0.0014	0.0005	0.0078	0.0014	0.0196	0.0005
chinese	0.0012	0.0006	0.0006	0.0006	0.0006	0.0517	0.0012	0.0006
food	0.0063	0.0004	0.0063	0.0004	0.0008	0.0020	0.0004	0.0004
lunch	0.0017	0.0006	0.0006	0.0006	0.0006	0.0011	0.0006	0.0006
spend	0.0012	0.0006	0.0012	0.0006	0.0006	0.0006	0.0006	0.0006

Smoothing moves probability mass from the seen bigrams to the unseen ones: C*(i want) falls from 827 to 527.1 and C*(to eat) from 686 to 429.8, while every former zero rises to a small positive value, so no bigram has probability 0 any more.

1.1 Why smoothing is needed

The unsmoothed (maximum-likelihood) estimate of a bigram is $P(w_n \mid w_{n-1}) = C(w_{n-1}\ w_n) / C(w_{n-1})$. A vocabulary of V = 1446 words allows 1446^2 = 2,090,916 different bigrams, but a corpus of 9,332 sentences can only show a small fraction of them, which is why most cells of Figure 4.1 are zero. Any sentence that contains a single unseen bigram then receives probability 0, its log-probability is minus infinity and its perplexity is undefined, even when the sentence is perfectly good English. Add-1 smoothing avoids this by pretending that every possible bigram was seen once more than it really was.

1.2 Denominators used for each row

Previous word w	C(w)	C(w) + V (denominator)
i	2533	3979
want	927	2373
to	2417	3863
eat	746	2192
chinese	158	1604
food	1093	2539
lunch	341	1787
spend	278	1724

All eight cells of one row share the same denominator, so a row of Table 2 is obtained by dividing (count + 1) by that number, and a row of Table 1 by multiplying the same ratio by C(w). For example, the row “want” uses 927 + 1446 = 2373: P*(i | want) = 3/2373 = 0.0013, P*(want | want) = 1/2373 = 0.0004, P*(to | want) = 609/2373 = 0.2566, P*(eat | want) = 2/2373 = 0.0008, P*(chinese | want) = P*(food | want) = 7/2373 = 0.0029, P*(lunch | want) = 6/2373 = 0.0025 and P*(spend | want) = 2/2373 = 0.0008. Multiplying each by C(want) = 927 gives the row of Table 1, e.g. C*(want to) = 609 × 927 / 2373 = 237.9.

1.3 Unsmoothed probabilities for comparison

Table 3. Unsmoothed (maximum-likelihood) probabilities P(column | row) = C / C(row)

	i	want	to	eat	chinese	food	lunch	spend
i	0.0020	0.3265	0.0000	0.0036	0.0000	0.0000	0.0000	0.0008
want	0.0022	0.0000	0.6559	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.0008	0.0000	0.0017	0.2838	0.0008	0.0000	0.0025	0.0873
eat	0.0000	0.0000	0.0027	0.0000	0.0214	0.0027	0.0563	0.0000
chinese	0.0063	0.0000	0.0000	0.0000	0.0000	0.5190	0.0063	0.0000
food	0.0137	0.0000	0.0137	0.0000	0.0009	0.0037	0.0000	0.0000
lunch	0.0059	0.0000	0.0000	0.0000	0.0000	0.0029	0.0000	0.0000
spend	0.0036	0.0000	0.0036	0.0000	0.0000	0.0000	0.0000	0.0000

1.4 Effect of smoothing on selected bigrams

Bigram	C	MLE P	Add-1 P*	C*	Discount C*/C
i want	827	0.3265	0.2081	527.1	0.637
want to	608	0.6559	0.2566	237.9	0.391
to eat	686	0.2838	0.1778	429.8	0.627
eat lunch	42	0.0563	0.0196	14.6	0.348
chinese food	82	0.5190	0.0517	8.2	0.100
food i	15	0.0137	0.0063	6.9	0.459
i to	0	0.0000	0.0003	0.6	–
to food	0	0.0000	0.0003	0.6	–

The discount C*/C shows how much the frequent bigrams pay for the zeros: “i want” keeps about 64% of its count, but “chinese food” (82 occurrences out of only 158 for “chinese”) drops to 8.2, a discount of 0.10, because the row of “chinese” has a small denominator (158 + 1446 = 1604) and so each of the 1446 pseudo-counts takes a large share. This is the main weakness of add-1 smoothing when V is large compared with the counts.

1.5 Effect on the probability of a sentence

For the sentence “i want to eat chinese food” the five bigrams of the sentence have the unsmoothed probabilities 0.3265, 0.6559, 0.2838, 0.0214 and 0.5190, whose product is 6.77e-04. With add-1 smoothing the factors become 0.2081, 0.2566, 0.1778, 0.0078 and 0.0517 and the product is 3.81e-06, much smaller because every factor lost mass. For the ungrammatical “i to eat”, the unsmoothed model gives P(to | i) × P(eat | to) = 0 × 0.2838 = 0, while the smoothed model gives 0.00025 × 0.1778 ≈ 4.5×10^{-5}. The ungrammatical sentence is therefore no longer ruled out completely, which is exactly what smoothing is meant to do.

1.6 Where does the probability mass go?

Row	Visible MLE mass	Visible P* mass	Mass left for the other 1438 words
i	0.3328	0.2139	0.7861
want	0.6785	0.2684	0.7316
to	0.3769	0.2379	0.7621
eat	0.0831	0.0319	0.9681
chinese	0.5316	0.0574	0.9426
food	0.0320	0.0169	0.9831
lunch	0.0088	0.0062	0.9938
spend	0.0072	0.0058	0.9942

Figure 4.1 shows only eight of the 1446 columns, so the visible probabilities of a row do not add up to 1. The table shows that after smoothing the eight visible cells hold less probability (for example 0.2139 instead of 0.3328 for “i”), and the rest of every row, which is shared by the other 1438 words, grows accordingly. Over the whole vocabulary each smoothed row still sums to exactly 1, because the denominator C(w) + V contains one pseudo-count for every possible next word.

1.7 Alternatives

Add-1 is simple and is a useful baseline, but for language modelling it is usually replaced by add-k with k < 1, by Good–Turing discounting, or by combining the bigram with a unigram model through backoff or interpolation. Kneser–Ney smoothing is the method most often used in practice for n-gram models.

Answer 2. Bag generation with a bigram grammar (Python)

Approach. The program first builds a bigram language model with add-1 smoothing from a training corpus (sentence markers <s> and </s> are added). Given a shuffled bag of words, it searches for the ordering with the highest probability $P(w_1|\text{<s>}) \cdot P(w_2|w_1) \cdots P(\text{</s>}|w_n)$. Because n words have n! orderings, it uses a beam search: it builds the sentence left to right, at each step tries every unused word, and keeps only the best 200 partial sentences. Log-probabilities are summed to avoid underflow. The corpus below is a small demonstration corpus; any larger text file can be used by passing its lines to Bigram().

```
import itertools, math
from collections import Counter
CORPUS = """i want to eat chinese food
i want to eat lunch
i want to spend money
i want chinese food
to eat lunch is nice
i want to eat
tell me about chinese food
i would like to eat lunch
i want food
give me chinese food
i want to spend lunch
want to eat chinese food
i am looking for chinese food
i want to eat food
list all chinese food
chinese food is good
i want to spend more money
we want to eat lunch today""".split("\n")

class Bigram:
    def __init__(self, sentences):
        self.uni, self.bi = Counter(), Counter()
        for s in sentences:
            w = ["<s>"] + s.lower().split() + ["</s>"]
            self.uni.update(w)
            self.bi.update(zip(w, w[1:]))
        self.V = len(self.uni)
    def logp(self, a, b):                       # add-1 smoothed log P(b | a)
        return math.log((self.bi[(a, b)] + 1) / (self.uni[a] + self.V))
    def score(self, seq):
        w = ["<s>"] + list(seq) + ["</s>"]
        return sum(self.logp(a, b) for a, b in zip(w, w[1:]))

def order(bag, lm, beam=200):
    """Guess the original order of a bag of words (beam search over bigram scores)."""
    bag = [w.lower() for w in bag]
    beams = [(0.0, ["<s>"], tuple(sorted(bag)))]
    for _ in bag:
        nxt = {}
        for sc, seq, rest in beams:
```

```
            for w in set(rest):
                r = list(rest); r.remove(w)
                key = (tuple(seq + [w]), tuple(r))
                nxt[key] = (sc + lm.logp(seq[-1], w), seq + [w], tuple(r))
        beams = sorted(nxt.values(), key=lambda x: -x[0])[:beam]
    best = max(beams, key=lambda b: b[0] + lm.logp(b[1][-1], "</s>"))
    return " ".join(best[1][1:])

if __name__ == "__main__":
    lm = Bigram(CORPUS)
    for bag in (["food", "want", "i", "chinese"],
                ["eat", "to", "lunch", "want", "i"],
                ["spend", "i", "money", "to", "want"],
                ["is", "good", "food", "chinese"]):
        print(sorted(bag), "->", order(bag, lm))
```

Output obtained by running the program:

```
['chinese', 'food', 'i', 'want'] -> i want chinese food
['eat', 'i', 'lunch', 'to', 'want'] -> i want to eat lunch
['i', 'money', 'spend', 'to', 'want'] -> i want to spend money
['chinese', 'food', 'good', 'is'] -> is good chinese food
```

The first three bags are restored exactly. The fourth comes out as “is good chinese food” instead of “chinese food is good”, because a bigram model only sees one word of context and the tiny corpus offers little evidence. A larger corpus, a trigram model (change the context to the last two words) or a larger beam would improve the guesses.

2.1 The search problem

For a bag of n words the program looks for the permutation that maximises the bigram probability of the whole sentence, including the start and end markers. The number of permutations grows as n!: 24 for 4 words, 120 for 5 words, 720 for 6 words, about 3.6 million for 10 words and about 1.3×10^{12} for 15 words, so listing them all is only possible for very short bags. The task can be seen as finding the best path through all the words in a weighted graph (similar to the travelling-salesman problem), which is why a heuristic search is used.

2.2 Algorithm step by step

1. **Training.** Add <s> and </s> to every training sentence, lowercase the text, and count unigrams and bigrams.
2. **Smoothing.** Estimate every bigram as (C(a b) + 1) / (C(a) + V), where V is the number of word types, so that unseen pairs of words never get probability 0.
3. **Initial state.** One partial sentence (<s>) with log-probability 0 and the whole bag as the set of unused words.
4. **Expansion.** For every partial sentence and every distinct unused word, append the word and add log P(word | previous word) to the score. Using set(rest) means a repeated word in the bag is tried once per step, not twice.
5. **Pruning.** Keep only the 200 best-scoring partial sentences (the beam).
6. **Finish.** After n steps, add log P(</s> | last word) to each complete sentence and return the best one.

Each step expands at most B × n candidates (B = beam width, n = bag size) and sorts them, so the whole search needs about $O(n^2 B \log(nB))$ time instead of O(n!).

2.3 Statistics of the demonstration model

The demonstration corpus has 18 sentences and 124 tokens including the markers; the vocabulary has V = 28 word types (including <s> and </s>) and 49 distinct bigrams, so 735 of the 784 possible bigrams are unseen and rely on smoothing.

2.4 Worked example: ranking all orderings of the first bag

For the first bag {food, want, i, chinese} there are only 24 orderings, so they can all be scored. The best one is built from the following smoothed bigram probabilities: P(i|<s>) = 12/46 = 0.2609; P(want|i) = 10/39 = 0.2564; P(chinese|want) = 2/39 = 0.0513; P(food|chinese) = 9/36 = 0.2500; P(</s>|food) = 10/38 = 0.2632. The product is 2.26e-04, which is a log-probability of -8.40. The table lists the best five and the worst ordering.

Rank	Ordering	Log-probability
1	i want chinese food	-8.40
2	chinese i want food	-12.39
3	i chinese want food	-12.90
4	i want food chinese	-12.90
5	want i chinese food	-13.18
24	food want i chinese	-18.38

The correct order is separated from the second-best one by almost 4 units of log-probability (a factor of more than 50), so even a very small beam finds it. The program's answer is the same as this exhaustive search.

2.5 Beam search compared with exhaustive search

Bag (sorted)	Original sentence	Beam (200)	Exhaustive	Exact?
chinese, food, i, want	i want chinese food	i want chinese food	i want chinese food	Yes
eat, i, lunch, to, want	i want to eat lunch	i want to eat lunch	i want to eat lunch	Yes
i, money, spend, to, want	i want to spend money	i want to spend money	i want to spend money	Yes
chinese, food, good, is	chinese food is good	is good chinese food	is good chinese food	No
chinese, eat, food, i, to, want	i want to eat chinese food	i want to eat chinese food	i want to eat chinese food	Yes
i, money, more, spend, to, want	i want to spend more money	i want to spend more money	i want to spend more money	Yes
eat, lunch, to, today, want, we	we want to eat lunch today	we want to eat lunch today	we want to eat lunch today	Yes
eat, is, lunch, nice, to	to eat lunch is nice	to eat lunch is nice	to eat lunch is nice	Yes

For all eight bags the beam search returned the same sentence as the exhaustive search over every permutation, so the search is not the weak point. Seven of the eight results equal the original sentence. All eight original sentences come from the training corpus itself, so this test shows that the program works but is partly memorisation; a fair evaluation needs sentences that are not in the training data.

2.6 Why the fourth bag fails

The two candidate orderings score -12.63 (“is good chinese food”) and -12.85 (“chinese food is good”), so the model genuinely prefers the wrong one. The reason is visible in the counts: </s> follows “food” in 9 of its 10 occurrences, which makes ending a sentence with “food” cheap, while “good” occurs only once in the corpus, so P(</s> | good) is only about 0.07. This also means that the closing remark above needs a correction: the error is a *model* error, not a *search* error, and enlarging the beam does not remove it (a beam of width 5 happens to return “chinese food is good”, but only because pruning happens to drop a partial sentence of the better-scoring wrong order). The reliable cures are a larger training corpus and a trigram model, which sees two words of context.

2.7 Limitations and improvements

- **Training data.** A 124-token corpus is far too small. Training on a real corpus (for example the Berkeley Restaurant corpus of 9,332 sentences) gives much more reliable probabilities.
- **Longer context.** A trigram model scores $P(w_n \mid w_{n-2}\, w_{n-1})$; in the code only the beam key and logp() need to be changed to use the last two words.
- **Better smoothing.** Add-1 gives too much mass to unseen pairs when V is large (see Answer 1); interpolation or Kneser–Ney smoothing is preferable.
- **Evaluation.** The output can be measured by the exact-match rate or by BLEU against the original sentence.
- **Unknown words.** A word never seen in training only receives the smoothing pseudo-count, so the program still runs but places such a word almost arbitrarily.

Answer 3. Penn Treebank POS tags (punctuation ignored)

Each word is written as word/TAG. Possessive ’s is split off as its own token (POS), as in the Treebank.

a. It/PRP is/VBZ a/DT nice/JJ night/NN

b. This/DT crap/NN game/NN is/VBZ over/IN a/DT garage/NN in/IN Fifty-second/JJ Street/NNP

c. Nobody/NN ever/RB takes/VBZ the/DT newspapers/NNS she/PRP sells/VBZ

d. He/PRP is/VBZ a/DT tall/JJ skinny/JJ guy/NN with/IN a/DT long/JJ sad/JJ mean-looking/JJ kisser/NN and/CC a/DT mournful/JJ voice/NN

e. I/PRP am/VBP sitting/VBG in/IN Mindy/NNP ’s/POS restaurant/NN putting/VBG on/RP the/DT gefillte/JJ fish/NN which/WDT is/VBZ a/DT dish/NN I/PRP am/VBP very/RB fond/JJ of/IN

f. When/WRB a/DT guy/NN and/CC a/DT doll/NN get/VBP to/TO taking/VBG peeks/NNS back/RB and/CC forth/RB at/IN each/DT other/JJ why/UH there/RB you/PRP are/VBP indeed/RB

Notes on the harder choices: “Nobody” is tagged NN (the Treebank treats it as a noun-like indefinite). “crap” modifies “game” so it is a noun (NN) used as a modifier. “Fifty-second” is an ordinal adjective (JJ) and “Street” belongs to a proper name (NNP). “newspapers she sells” is a reduced relative clause, so “she” is PRP and “sells” VBZ. “putting on” is a phrasal verb with “on” as a particle (RP), not a preposition. “gefillte” is a foreign-origin modifier tagged JJ, and “which” is

a wh-determiner/pronoun (WDT). “of” stays IN even though it is stranded at the end of the clause. In (f), “When” is WRB, “get to taking” is VBP + TO + VBG, “back and forth” are adverbs (RB), and “why” is an interjection here (UH), which is debatable and could also be tagged WRB.

3.1 Method used

1. **Tokenise** the sentence and split clitics, so that Mindy’s becomes Mindy + ’s (NNP + POS) as the Treebank does; punctuation is ignored as the question allows.
2. **Look up the usual tag** of each word (for example the, a: DT; is: VBZ; and: CC).
3. **Resolve ambiguity from the context.** After a determiner a word is an adjective or noun; to followed by a base verb is TO, while to followed by a noun phrase is IN; a verb after a 3rd-person singular subject is VBZ, after I or you it is VBP.
4. **Check against the Treebank guidelines** for special cases such as particles (RP), indefinite pronouns and wh-words.

A statistical tagger would make the same choices by taking the tag sequence that maximises the product of P(tag | previous tag) × P(word | tag), found with the Viterbi algorithm; the manual rules above are the linguistic knowledge that such a tagger learns from its training data.

3.2 Tags used and their meaning

Tag	Meaning	Examples from the sentences
PRP	personal pronoun	It, He, I, she, you
VBZ	verb, 3rd person singular present	is, takes, sells
VBP	verb, non-3rd person present	am, get, are
VBG	verb, gerund or present participle	sitting, putting, taking
DT	determiner	a, the, This, each
JJ	adjective	nice, tall, mournful, fond
NN	noun, singular or mass	night, game, Nobody, kisser
NNS	noun, plural	newspapers, peeks
NNP	proper noun, singular	Street, Mindy
IN	preposition or subordinating conjunction	over, in, with, at, of
RB	adverb	ever, very, back, forth, indeed
CC	coordinating conjunction	and
RP	particle	on (putting on)
WDT	wh-determiner	which
WRB	wh-adverb	When
TO	the word to	to (get to taking)
UH	interjection	why
POS	possessive ending	’s

3.3 Sentence-by-sentence analysis

a. a subject pronoun (It/PRP), a linking verb (is/VBZ) and a noun phrase made of a determiner, an adjective and a noun (a nice night).

b. the subject “This crap game” is a noun phrase in which crap is a noun modifier. The predicate is a copular clause with two prepositional phrases, “over a garage” and “in Fifty-second Street”; the second contains the proper name Fifty-second Street.

c. the main clause is “Nobody ever takes the newspapers”; “she sells” is a relative clause that modifies newspapers and has no relative pronoun (the object is missing), so she is an ordinary PRP and sells a VBZ.

d. a long noun phrase after “with”: a coordinated list of three adjectives (long, sad, mean-looking) before kisser, then a second noun phrase “a mournful voice” joined by and (CC). The compound adjective mean-looking is tagged JJ as a whole.

e. the progressive “am sitting” (VBP + VBG), the phrasal verb “putting on” whose particle on is RP, a direct object “the gefillte fish”, and the relative clause “which is a dish I am very fond of”, where which is WDT and the preposition of is stranded at the end.

f. the subordinate clause “When a guy and a doll get to taking peeks…” has a coordinated subject and the verb get (VBP); to is TO and taking is a gerund. The main clause “why there you are indeed” is a fixed expression in which why is tagged UH.

3.4 The ambiguous words and why each tag was chosen

Word	Chosen	Possible alternative	Reason
Nobody	NN	PRP, NNP	the Treebank tags nobody, somebody and anybody as NN
crap	NN	JJ	it is a noun used to modify game (noun compound)
sells	VBZ	NNS	it follows a pronoun and is the verb of the relative clause
putting on	VBG + RP	IN	on can be dropped from the phrase and the verb still takes the object, so it is a particle
gefillte	JJ	FW	an adjective in the English sentence, even though the word is borrowed
of	IN	RB	a preposition even when stranded at the end of the clause
to (get to taking)	TO	IN	it is followed by a verb form, not by a noun phrase
back, forth	RB	NN, JJ	they function as adverbs in the phrase “back and forth”
each other	DT + JJ	PRP	the Treebank tags each as DT and other as JJ
why	UH	WRB	here it is not a question word but an interjection

3.5 Tag statistics

The six sentences contain 80 tagged tokens (a: 5, b: 10, c: 7, d: 16, e: 21, f: 21), including the separated ’s. The most frequent tags are NN (13), DT (12), JJ (11), IN (6), PRP (6), RB (6); 18 different tags are used altogether. Nouns of all kinds (NN, NNS, NNP) account for 17 tokens and verbs (VBZ, VBP, VBG) for 13.

Overall summary of the assignment

Question 1 gives the add-1 smoothed counts and probabilities for the eight words of Figure 4.1, with the denominators, the unsmoothed comparison, and the effect on seen and unseen bigrams. Question 2 provides a working bag-generation program with a smoothed bigram model and beam search, tested against exhaustive search, with an analysis of the one failure. Question 3 tags the six sentences with the Penn Treebank tagset and explains the difficult choices.